

МИНОБРНАУКИ РОССИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«САНКТ-ПЕТЕРБУРГСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ ПЕТРА
ВЕЛИКОГО»**

Институт компьютерных наук и кибербезопасности

Высшая школа технологий искусственного интеллекта

Направление 02.03.01 Математика и компьютерные науки

Отчёт по дисциплине «Математическая логика и теория автоматов»

Лабораторная работа №5 «Реализация LL(1)
анализатора»

Вариант: 18

Студент: _____

Черданцев Артём Сергеевич

Преподаватель: _____

Востров Алексей Владимирович

«____» _____ 20__ г.

Санкт-Петербург, 2025

Содержание

Введение	3
1 Математическое описание	4
1.1 LL(k)-грамматики	4
1.2 Грамматика	4
1.2.1 Анализ грамматики	4
1.2.2 Семантические действия	5
1.3 Пример дерева для грамматики	6
2 Особенности реализации	7
2.1 Структура данных	7
2.1.1 lookupTable	7
2.1.2 alternatives	7
2.1.3 stack	7
2.2 Основные функции	7
2.2.1 parse	7
2.2.2 generate	8
3 Результаты работы программы	10
Заключение	12
Список источников	13

Введение

Дана следующая грамматика: $G ::$

$$\begin{aligned} S' &\rightarrow S\$ \\ S &\rightarrow BA \\ A &\rightarrow +BA|\varepsilon \\ B &\rightarrow DC \\ C &\rightarrow *DC|\varepsilon \\ D &\rightarrow (S)|a \end{aligned}$$

Требуется построить множества FIRST и FOLLOW для каждого нетерминала грамматики и таблицу выбора (lookup table). На их основе реализовать (не)детерминированный левый анализатор (проверка принадлежности цепочки грамматике). Назначить семантические действия части заданных продукций (например, генерация машинноориентированного кода).

1 Математическое описание

1.1 LL(k)-грамматики

Порождающей грамматикой Хомского G называется множество: $\{T, N, S, R\}$, где:

- T - конечное множество символов (терминальный словарь);
- N - конечное множество символов (нетерминальный словарь);
- $S \in N$ - начальный нетерминал;
- R - конечное множество правил (продукций) вида $\alpha \rightarrow \beta$, где α и β цепочки над словарём $T \cup N$.

Контекстно-свободной грамматикой называется грамматика, у которой левые части всех продукций являются одиночными нетерминалами. То есть все правила грамматики имеют вид:

$$A \rightarrow \lambda, \text{ где } A \in T, \lambda \in N$$

Промежуточное состояние в процессе левого канонического вывода состоит из цепочки терминалов ω и самого левого нетерминала A и недовыведенной части x . Таким образом ωAx — текущая сентенциальная форма, ω — распознаваемая цепочка, ω — распознанная часть цепочки.

Грамматика имеет свойство LL(k), если для выбора правила достаточно рассмотреть только Ax и первые k символов непросмотренной цепочки x . Свойства LL(k)-грамматик:

- входная цепочка читается слева направо;
- строится левый вывод входной цепочки;
- на каждом шаге для принятия решения используется следующий символ непрочитанной части входной цепочки.

$FIRST(\alpha)$ — множество терминальных символов, с которых могут начинаться цепочки, выводимые из α .

$$FIRST(\alpha) = \{c \in T | \alpha \rightarrow^* c\beta\} \cup \{\varepsilon | \alpha \rightarrow^* \varepsilon\}$$
$$FIRST(A) = \bigcup_i FIRST\alpha_i, \text{ где } A \rightarrow \alpha_1 | \dots | \alpha_k \text{ — все альтернативы нетерминала } A.$$

$FOLLOW(A)$ — множество терминальных символов, с которых могут начинаться цепочки, которые следуют за A в любых сентенциальных формах.

$$FIRST(A) = \{c \in T | \alpha \rightarrow^* \alpha A c \beta\} \cup \{\$ | S \rightarrow^* \alpha A\}$$

Если $FIRST(\alpha)$ и $FIRST(\gamma)$ не пересекаются, то можно однозначно установить выбор между правилами $A \rightarrow \alpha$ и $A \rightarrow \gamma$ ($\alpha, \gamma \neq \varepsilon$). Если $FIRST(\alpha)$ и $FOLLOW(A)$ не пересекаются, то можно однозначно установить выбор между правилами $A \rightarrow \alpha$ и $A \rightarrow \varepsilon$.

КС-грамматика называется грамматикой с левой рекурсией, если в ней существует вывод $A \rightarrow^+ A\beta$.

1.2 Грамматика

1.2.1 Анализ грамматики

Дана следующая грамматика: G ::

$$\begin{aligned}
S' &\rightarrow S\$ \\
S &\rightarrow BA \\
A &\rightarrow +BA \mid \varepsilon \\
B &\rightarrow DC \\
C &\rightarrow *DC \mid \varepsilon \\
D &\rightarrow (S) \mid a
\end{aligned}$$

Эта грамматика не содержит циклов, а все нетерминалы достижимы.

Множества FIRST и FOLLOW:

	FIRST	FOLLOW
S'	$\{ (, a \}$	$\{ \$ \}$
S	$\{ (, a \}$	$\{ \$,) \}$
A	$\{ + \}$	$\{ \$,) \}$
B	$\{ (, a \}$	$\{ \$, +,) \}$
C	$\{ + \}$	$\{ \$, +,) \}$
D	$\{ (, a \}$	$\{ \$, +, *,) \}$

Таблица выбора (lookup table) это таблица, с помощью которой можно однозначно определить, какую продукцию применять, исходя из поданных на вход терминала и нетерминала. Если в таблице не возникает конфликтов (две продукции для выбора одного терминала или используются не все продукции), тогда грамматика является LL(1)-грамматикой.

Построим для данной грамматики таблицу выбора (lookup table):

	a	(	)	+	*	\$
S'	4	4	3	-	-	-
S	-	-	-	-	-	-
A	-	-	-	1	-	3
B	-	-	-	-	-	-
C	-	-	-	-	2	-
D	0	-	-	-	-	-

В полученной таблице используются все продукции. Таким образом можно подтвердить, что грамматика является LL(1). Цифры представленные в таблице - семантические действия.

1.2.2 Семантические действия

Список семантических действий:

- 0 - pull a (чтение значения для a)
- 1 - add (сложение двух значений)
- 2 - mul (умножение двух значений)
- 3 - close (конец работы со строкой)
- 4 - open (начало работы со строкой)

Семантические действия в данном случае - это код для некоторого алгоритма, обозначающий, какие действия совершать со строкой.

Пример: $(a_1 + a_2) * a_3 \$$

Семантические действия для данной строки: open pull pull add pull mul close

Эти действия обозначают следующий алгоритм действий:

1. Начало работы со строкой
2. Чтение значения для a_1
3. Чтение значения для a_2
4. Сложение этих двух значений
5. Чтение значения для a_3
6. Умножение его и результата из шага 4
7. Конец работы со строкой

1.3 Пример дерева для грамматики

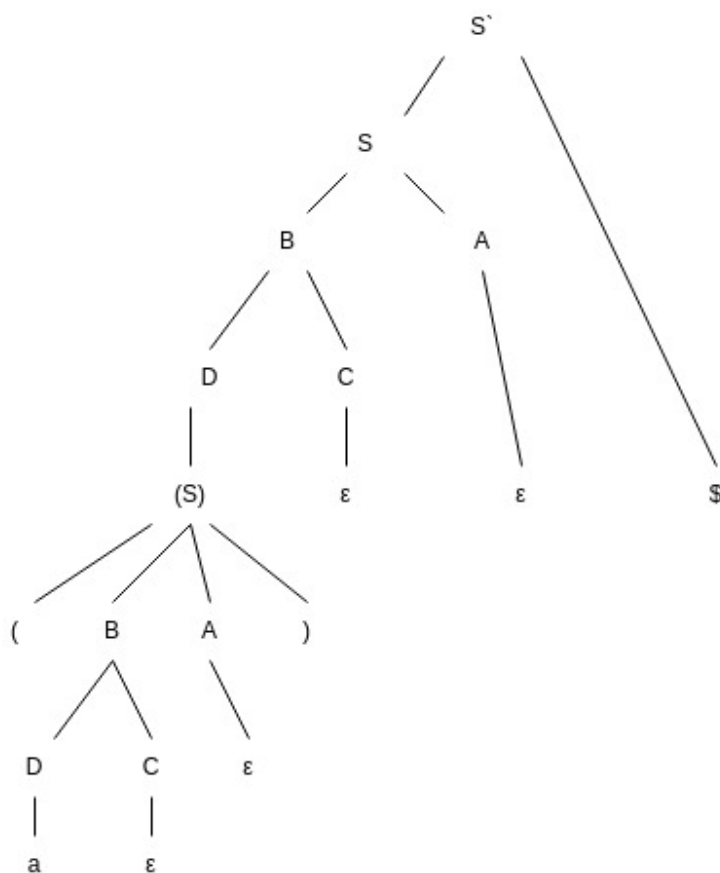


Рис. 1: Пример дерева для построения строки '(a)'

2 Особенности реализации

2.1 Структура данных

2.1.1 lookupTable

Это трёхмерный массив, представляющий таблицу разбора, где первая размерность — это индекс нетерминалов, вторая — терминалов, а третья — правая часть продукции (множество правил), которые заменяют нетерминалы в строке.

```
1 final char[][][] lookupTable = new char[NONTERMINALS.length][TERMINALS.length][];
```

2.1.2 alternatives

Эта структура хранит список терминалов, которые связаны с нетерминалом. Это необходимо для генерации цепочек, когда нужно случайным образом выбрать одно из возможных правил для нетерминала.

```
1 final Map<Character, List<Character>> alternatives = new TreeMap<>();
```

2.1.3 stack

Используется как основной механизм для хранения нетерминалов и терминалов во время разбора или генерации строк.

```
1 Deque<Character> stack = new ArrayDeque<>();
```

2.2 Основные функции

2.2.1 parse

Функция принимает строку на вход и пытается разобрать её в соответствии с грамматикой. Использует стек для замены нетерминалов на их продукции и проверяет соответствие входной строки грамматике. Возвращает объект ParseResult с результатом разбора и семантикой.

Алгоритм основан на LL(1) разборе:

- Если символы стека и входной строки совпадают — они извлекаются.
- Если на вершине стека терминал — строка отвергается.
- Если найдено семантическое действие — оно добавляется в список значений.
- Если продукция не найдена — строка отвергается.

Вход: строка для анализа.

Выход: результат анализа, включает вердикт (Verdict) и семантические значения.

```
1 public ParseResult parse(String input) {  
2     Deque<Character> stack = new ArrayDeque<>();  
3     stack.push(END_OF_INPUT);  
4     stack.push(START_NONTERMINAL);
```

```

5     Deque<Character> inputS = new ArrayDeque<>();
6     List<String> meaning = new ArrayList<>();
7     for (int i = 0; i < input.length(); i++) {
8         char c = input.charAt(i);
9         if (!isTerminal(c) || c == END_OF_INPUT) {
10             return new ParseResult(Verdict.UnknownSymbol, null);
11         }
12         inputS.add(c);
13     }
14     inputS.add(END_OF_INPUT);
15     while (!stack.isEmpty() && !inputS.isEmpty()) {
16         if (inputS.getFirst() == stack.getFirst()) {
17             stack.pop();
18             inputS.pop();
19             continue;
20         }
21         if (isTerminal(stack.getFirst())) {
22             // Unmatched terminal found
23             return new ParseResult(Verdict.Rejected, null);
24         }
25         if (isSemantic(stack.getFirst())) {
26             meaning.add(SEMANTICS.get(stack.pop()));
27             continue;
28         }
29         var rightHandSide = lookup(stack.pop(), inputS.getFirst());
30         if (rightHandSide == null) {
31             //No production found in parse table
32             return new ParseResult(Verdict.Rejected, null);
33         }
34         for (char c : rightHandSide) {
35             stack.push(c);
36         }
37     }
38     return new ParseResult(Verdict.Accepted, meaning);
39 }

```

2.2.2 generate

Функция генерирует случайную строку на основе грамматики. Она работает со стеком и случайно выбирает одно из доступных правил для каждого нетерминала, постепенно заменяя нетерминалы на терминалы. В результате возвращается строка, которая соответствует грамматике.

Вход: productions грамматики.

Выход: случайная строка, соответствующая грамматике.

```

1 public String generate() {
2     Deque<Character> stack = new ArrayDeque<>();
3     stack.push(START_NONTERMINAL);

```



```

4      StringBuilder sb = new StringBuilder();
5      while (!stack.isEmpty()) {
6          if (isTerminal(stack.getFirst())) {
7              sb.append(stack.pop());
8              continue;
9          }
10         if (isSemantic(stack.getFirst())) {
11             stack.pop();
12             continue;
13         }
14         char nonTerminal = stack.pop();
15         var terminals = alternatives.get(nonTerminal);
16         for (char c : lookup(nonTerminal,
17             ↪ terminals.get(random.nextInt(terminals.size())))) {
18             stack.push(c);
19         }
20     }
21     return sb.toString();
22 }

```

3 Результаты работы программы

На рис. 1 представлена генерация 3-х случайных строк, соответствующих грамматике

```
Выберите вариант: 2
////////////////////
Случайная подходящая цепочка: (((a)))$
////////////////////

Выберите вариант: 2
////////////////////
Случайная подходящая цепочка: ((((((a))*((a)+a*(((a+a*a))*((a+(a)+(a+a))+((a)*a+(a)))*(a)*((a*a)+a+a+a))))))$
////////////////////

Выберите вариант: 2
////////////////////
Случайная подходящая цепочка: (((a)*a))$
////////////////////
```

Рис. 2: Генерация случайны строк, соответствующих грамматике

На рис. 2 представлена проверка валидной строки для этой грамматики.

```
Выберите вариант: 1
////////////////////
Введите строку для проверки: a*a+a+(a+(a))$
Цепочка удовлетворяет грамматике. Семантика:
open pull pull mul pull add pull pull add add close
////////////////////
```

Рис. 3: Успешная проверка строки

На рис. 3 представлен вывод ошибок при вводе невалидных строк.

```
Выберите вариант: 1
////////////////////
Введите строку для проверки: a*a+a+(a+(a))
Цепочка НЕ удовлетворяет грамматике.
////////////////////

Выберите вариант: 1
////////////////////
Введите строку для проверки: фывф
Цепочка содержит символы не из алфавита.
////////////////////
```

Рис. 4: Вывод ошибок невалидных строк

На рис. 3 представлен вывод ошибки при вводе пустой строки.

```
Выберите вариант: 1
////////////////////////
Введите строку для проверки:
Пустая строка не удовлетворяет грамматике!
////////////////////////
```

Рис. 5: Вывод ошибки пустой строки

Заключение

В ходе выполнения работы был разработан детерминированный синтаксический анализатор, реализующий алгоритм LL(1)-разбора для заданной контекстно-свободной грамматики.

Разработанный анализатор позволяет не только проверять принадлежность строки грамматике, но и выполнять семантические действия, ассоциированные с определёнными продукциями. Это делает возможной генерацию атрибутивного вывода, включая, например, машинно-ориентированный код или промежуточные представления.

Также реализована функция генерации строк, соответствующих грамматике, с помощью случайного выбора продукций.

Преимущества:

- система построена таким образом, что может быть адаптирована под любую LL(1)-грамматику — достаточно изменить исходные продукции,
- возможность привязки семантических действий к конкретным правилам

Недостатки:

- Программа не поддерживает нетерминалы и терминалы, состоящие из нескольких символов

Масштабирование:

- Поддержка многосимвольных нетерминалов и терминалов.
- Расширение функционала до LL(k)-анализатора.

Список источников

1. Ахо, А. Компиляторы. Принципы, технологии, инструменты. – СПб.: Вильямс, 2003. – С. 1178.
2. Карпов, Ю.Г. Теория и технология программирования. Основы построения трансляторов. – СПб.: БХВ-Петербург, 2005. – С. 272.